1 OS structures

Which of the following is TRUE for a microkernel operating system?

- Linux is a representative example of microkernel operating systems
- It is less reliable than a monolithic kernel operating system.
- It consists of a small kernel that provides basic services such as scheduling and inter-process communication, with additional services implemented as user-space processes or servers.
- It is a single-layer architecture where all operating system services are implemented within the kernel space.

Maximum marks: 1

2 Processes and threads

Which of the following statements about process states is TRUE?

- A process can only be in one state at a time.
- A process enters the "blocked" state if its execution is done.
- A process can be in multiple states simultaneously.
- All processes start in the "running" state.

Which statement about processes in operating systems is FALSE?

- A process is an instance of a program in execution.
- Processes are independent and can't communicate with each other.
- Context switching is the mechanism used to switch between different processes.
- Processes have their own memory space and resources allocated by the operating system.

Which of the following statements accurately highlights a difference between processes and threads?

- Processes run independently and are isolated, whereas threads are designed to run as parts of a process.
- Processes can be created faster than threads because they require less overhead.
- Threads have their own separate memory space, while processes share a common memory space.
- Threads share CPU and system resources, while processes do not share these resources with other processes.

Maximum marks: 3

3 Scheduling

Task	Arrival Time	Execution Time
T1	0	10
T2	5	12
T3	5	15
T4	10	12

Given the above task set, which scheduling algorithm performs WORST in terms of turnaround time?

- Round-Robin with time quantum=5
- Shortest time to completion first
- Shortest job first
- FIFO

What is a characteristic of the Completely Fair Scheduler (CFS)?

- It divides the CPU time into fixed time slices and assigns them to processes based on their fixed priority level.
- It aims to provide fair distribution of CPU time among processes using virtual runtime.
- It prioritizes processes based on their arrival time, scheduling the oldest process first.
- It assigns a static priority to each process and uses preemptive scheduling.

```
void
scheduler(void)
{
  struct proc *p;
  struct cpu *c = mycpu();

  c->proc = 0;
  for(;;){
    // Avoid deadlock by ensuring that devices can interrupt.
    intr_on();
    // !!!(1)
    for(p = proc; p < &proc[NPROC]; p++) {
      acquire(&p->lock);
      if(p->state == RUNNABLE) {
        // Switch to chosen process.  It is the process's job
        // to release its lock and then reacquire it
        // before jumping back to us.
        // !!!(2)
        p->state = RUNNING;
        c->proc = p;
        swtch(&c->context, &p->context);

        // Process is done running for now.
        // It should have changed its p->state before coming back.
        c->proc = 0;
      }
      // !!!(3)
      release(&p->lock);
      // !!!(4)
    }
  }
}
```

The "scheduler" function is responsible for implementing the scheduling algorithm in xv6. Where should you start modifying and adding the core code to determine which process should be executed at a given time? There are four places with a comment like "!!!(1)". Select the right place to add your code.

- 4
- 2
- 1
- 3

Maximum marks: 3

4 Memory

Which of the following statements is TRUE regarding a virtual address and a physical address in computer systems?

- A virtual address is always larger than a physical address.
- A virtual address is generated by the CPU during program execution, while a physical address is assigned by the operating system to map to a specific location in memory.
- A virtual address represents a location in main memory, while a physical address represents a location in disk.
- A virtual address is used by the operating system to access hardware devices, while a physical address is used by applications to access system resources.

Which of the following statements is TRUE about address space?

1. The address space is an abstraction of a process's view of memory.
2. The heap segment is used to store function arguments and local variables.
3. The code segment is a static segment within address space
4. Data on a stack is removed in a Last-In-First-Out manner

- 1,3,4
- 1,3
- 3,4
- 2,3,4

Maximum marks: 2

5 Paging

Which of the following statements about paging is TRUE?

1. Paging can increase the processing speed of the physical memory

2. Multi-level paging can improve page access speed

3. Paging suffers from internal fragmentation

4. In a virtual memory system, a page and a page frame have the same size

- 2, 3
- 2, 3, 4
- 3, 4
- 1, 2, 3

What role does the Translation Lookaside Buffer (TLB) play in the paging process?

- It manages the allocation of physical memory frames to virtual memory pages, ensuring efficient memory utilization.
- It performs address translation for virtual memory pages without the need for page tables, eliminating the overhead of memory mapping.
- It stores copies of frequently accessed pages from memory to reduce disk I/O latency.
- It caches recently used page table entries to accelerate the translation of virtual addresses to physical addresses.

xv6 supports 64-bit virtual address and the page size by default is 4KB. How many bits are used for the default page offset? If the page size is reduced to 1KB, how many bits are used for page offset?

- 16, 14
- 12, 11
- 12, 10
- 10, 8

Maximum marks: 3

6 Concurrency

```
lock L
condition_variable cv

int producer(){
   grab L
   while buffer is full:
      cv.wait(L)
   add item to buffer
   cv.signal()
   release L
}

int consumer():
   grab L
   while buffer is empty:
      cv.wait(L)
   remove item from buffer
   cv.signal()
   release L
}
```

The pseudocode above implements a solution for the producer-consumer problem, where a condition variable is used.

What does the condition variable "cv" mainly achieve?

- It provides a mechanism for the producer to wait until there is space and for the consumer to wait until there is an item in the buffer.
- It increases the processing speed by allowing multiple producers and consumers to modify the buffer simultaneously.
- It locks the shared resource buffer to ensure that only one thread can access the buffer at a time.
- It ensures that the producer and consumer never run in parallel, thereby avoiding race conditions.

Why do we need a lock "L" here?

- To allow multiple threads to access the buffer simultaneously, increasing throughput.
- To manage the scheduling of threads so that the producer always runs before the consumer.
- To prevent the producer from executing before the consumer.
- To ensure that only one thread can add or remove items from the buffer at any given time.

Does the pseudocode work correctly with multiple consumers? And WHY?

- Yes. It works perfectly with more than one consumers.
- No. The producer may overwrite its own data in the buffer
- No. Consumers will take the same item
- No. It may freeze because one consumer can signal another consumer even when the buffer is empty.

If we want to implement a good solution for the producer-consumer problem using semaphores, how should we declare and initialize semaphores? Assume the buffer size is MAX and multiple producers and consumers should be supported. A lock can be added to prevent race condition.

- Two semaphores: one initialized to 0 which the producer waits on, and another initialized to MAX, which the consumer waits on.
- One semaphore which is initialized to 0.
- Two semaphores: one initialized to MAX which the producer waits on, and another initialized to 0, which the consumer waits on.
- One semaphore which is initialized to MAX.

Which of the following statements is TRUE about semaphores?

- If we implement a binary lock with a semaphore, it should be initialized to 0.
- If the semaphore value becomes 0 after a wait operation, the caller will sleep
- Semaphores have three operations, wait, post, and ready.
- All statements are false

Maximum marks: 5

7 I/O Devices

Which of the following components does NOT belong to the interface components of I/O devices

- Data register
- Status register
- Micro-controller
- Command register

What is the primary purpose of Direct Memory Access (DMA)?

- To bypass the CPU for certain operations, allowing peripheral devices to access memory directly.
- To allow the CPU to manage all memory operations directly.
- To increase the storage capacity of the system.
- To enhance the processing power of peripheral devices.

Which of the following statements is FALSE about HDDs?

- HDDs with higher RPM are faster
- The fundamental processing units in HDDs are sectors
- HDDs are byte-addressable.
- An HDD can have multiple platters

Why might an SSD have a shorter lifetime than an HDD under intensive write operations?

- Write operations of SSDs are slower than those on HDDs.
- SSDs degrade due to mechanical wear-out.
- SSDs require frequent defragmentation which shortens their lifespan.
- Cells in an SSD can only be written to a limited number of times.

Maximum marks: 4

8 File Systems

```
struct proc {
  uint sz;                     // Size of process memory (bytes)
  pde_t* pgdir;                // Page table
  char *kstack;                // Bottom of kernel stack for this process
  enum procstate state;        // Process state
  volatile int pid;            // Process ID
  struct proc *parent;         // Parent process
  struct trapframe *tf;        // Trap frame for current syscall
  struct context *context;     // swtch() here to run process
  void *chan;                  // If non-zero, sleeping on chan
  int killed;                  // If non-zero, have been killed
  struct file *ofile[NOFILE];  // Open files
  struct inode *cwd;           // Current directory
  struct shared *shared;       // Shared memory record (0 -> none)
  char name[16];               // Process name (debugging)
};
```

The above code shows the "proc" data structure of xv6. Within it, there is a **struct for "file"**.
1) **ofile[]** stores pointers to open files for this process
2) **struct file** stores paths of all open files
3) **struct file** stores inodes of all open files
4) **struct file** only stores the open file information for this process
5) **struct file** only stores the open file information for all processes currently running
Which of the following statements is CORRECT?

- 1 and 4
- 2 and 4
- 1 and 5
- 3 and 4

$$\mathbf{f = open(”/foo/test.txt”)}$$

The above code opens a file named test.txt. Which of the following is CORRECT?

- f=0, if this is the first opened file.
- f stores an inode number.
- f=3, if this is the first opened file.
- f stores the path name "/foo/test.txt"

To open the file test.txt, what would be the correct order to access the different on-disk structures?

1. inode of test.txt
2. inode of foo
3. inode of /
4. data block of test.txt
5. data block of foo
6. data block of /

- 3, 6, 2, 5, 1, 4
- 2, 5, 1
- 2, 5, 1, 4
- 3, 6, 2, 5, 1

Maximum marks: 3

9 Processes

```
int main() {

    fork()? fork():printf("success\n");

return 0;
}
```

The above code uses a ternary operator X ? A : B
It evaluates the value of X. If X is TRUE or greater than 0, A will be the output of this ternary operator. If X is 0 or negative, B will be the output.

How many processes will be generated by executing this code snippet, excluding the original parent process? [ ] **. (1 point)**

Will it print out "success" at the end? [ ] **(Please type yes or no)**

(1 point)

How many "success" will be printed out? [ ] **(1 point)**

If it will print out "success" at the end, which process will do it? All processes, parent, child, or NO (NO means it will not be printed out). [ ]

(1 point)

Maximum marks: 4

10 Scheduling

You implemented an MLFQ algorithm in xv 6. (Each blank is 1 point)

To prevent a task from getting starvation, MLFQ needs to [ ] **periodically.**

Assume that a task set is given as below:

Task	Arrival time	Execution time
T1	0	45
T2	0	20
T3	0	55

If an MLFQ algorithm with three queues is used to schedule the task set,
each queue is scheduled using Round-Robin with time slice of 10 ms, 20 ms, and 40 ms, respectively, from the high priority queue to low priority queue. Note that these tasks are enqueued in the order of A B C, i.e., although all tasks have the same arrival time, A is the first added to the queue, and then B, finally C.

Queue 1, high priority, 10ms
Queue 2, medium priority, 20ms
Queue 3, low priority, 40ms

What is the turnaround time of all tasks? Assume that the MLFQ does not use the method answered in the first question. Each task is allowed to execute only one time slice at each queue. (each blank is 1 point)

T1: [ ] **. It is finished at queue** [ ] **(Queue number)**

T2: [ ] **. It is finished at queue** [ ] **(Queue number)**

T3: [ ] **. It is finished at queue placeholder (Queue number)**

In this implementation, all queues in the MLFQ uses the same scheduling algorithm. Must all queues from MLFQ use the same scheduling algorithm? [ ] **(1 points)**

Maximum marks: 7

11 Memory

What is the segmentation method of memory management in OSs? Explain how it works and what hardware it needs. (3 points)

What is the problem of the segmentation? (1 point)

What method can we use to address the issue answered in the previous question? (1 point)

Maximum marks: 5

12 Paging

xv6-riscv we used for labs supports 64bit virtual address, but it does not need to use all bits for paging. Given that page size is 4KB and the size of page table entry is 8 Bytes, xv6 only uses 39 bits out of 64 bits for paging, where a 3-level paging is implemented. (Each blank is 1 point)

How many bits are used for one level of the 3-level page directory index? _____ **(1 point)**

Please divide the following virtual address into different parts. L1 index bits are at the most significant bits of the 39 bits. The answers should be in binary. (Each blank is 1 point)

Virtual address	L1 Index	L2 Index	L3 Index	Page offset
0x23059C20AF				

When it only uses 39 bits for virtual address, how large memory can this virtual address index? (1 point)

_____

We use this paging system to access pages of a given workload with a page replacement policy. It is found that if we increase the size of physical memory, the system has more page faults than before. Which replacement policy is used? (1 point)

_____

If the system is expected to execute a lot of loop sequential workloads in the future, which replacement algorithm should be used? (1 point)

_____

Maximum marks: 8

13 I/O Devices

Given an HDD spec as follows:

Model: UNKNOW HDD 4000
RPM: 7200
Average seek time: 9ms
Transfer speed: up to 160MB/s

If we want to read a file of 100MB, how long does it take to read the file? (Each blank is 1 point)

Average rotation latency: [ ] **ms (Report only the integer part)**

The transfer latency: [ ] **ms (Report only the integer part)**

SSDs are much faster than HDDs. The basic element in SSDs is a cell which can store 1 or a couple of bits. Then, how many bits can a multi-level cell store? (1 point)

[ ]

Writing to an SSD requires two operations. What are the two operations? Answer them in the correct order. (Each blank is 1 point)

[ ] [ ]

Maximum marks: 5